

Mensajes, roles y contexto en varios turnos

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 7 de 12 · Notas en español (es-ES)

Una llamada a `messages.create` es un viaje HTTP. Claude no recuerda el turno anterior salvo que **volváis a enviar esa historia**. Los notebooks de Vijay construían una lista de objetos `{role, content}`, añadían el último texto del asistente y preguntaban de nuevo. Esa lista es el **contexto** conversacional que posee vuestra aplicación.

Roles: user, assistant y system

En clase, **user** es el mensaje humano y **assistant** la salida del modelo que reinyectáis. Esos dos viven en el array `messages`. **System** no iba como tercer rol dentro de ese array en el patrón de Vijay: es un **parámetro aparte** de `create` (campo `system`). Palabras que señaló para el examen: cómo se configuran `messages` y el `system prompt`. `System` son instrucciones de conducta; `messages` son el diálogo.

Pieza	Dónde va	Función
user	<code>messages[]</code>	Pregunta actual o entrada de la app
assistant	<code>messages[]</code>	Respuestas previas que decidís conservar
system	<code>system</code> en <code>create</code>	Instrucciones permanentes, rol, normas anti-alucinación
tools / RAG (luego)	parámetros de <code>create</code> / texto recuperado	También ocupan la ventana de contexto de entrada

`System` es el análogo en API de un «archivo de instrucciones» en Claude Code / VS Code, pero hay que pasarlo **en cada petición** porque el servidor no guarda estado. Repetirlo cuesta tokens de entrada; la caché de prompts evita pagar el precio completo de un prefijo estable (módulo 09).

Por qué «¿cómo me llamo?» falla sin historia

Demo 1: el usuario dice «Hola, me llamo Vijay». Demo 2 (nuevo `create`, solo «¿cuánto es 1+1?»): aritmética, sin nombre. Demo 3 (solo «¿cómo me llamo?»): el modelo no tiene la línea previa salvo que la incluyáis. Demo 4: `messages = [presentación, saludo del asistente, 1+1, «2», «¿cómo me llamo?»]` — el nombre está en contexto y puede recuperarse.

ChatGPT, Claude Chat y Claude Code parecen con estado porque **esos productos** guardan la sesión y la reenvían. Vuestro notebook es una aplicación de IA: decidís RAM, base de datos o descartar. Las listas fijas son didácticas; en producción hay un bucle que hace `append`.

Métodos auxiliares que reutilizó Vijay

1. `add_user_message(messages, text)` — añade `role: user`.

2. `add_assistant_message(messages, text)` — añade la respuesta anterior para que la siguiente llamada la vea.
3. `chat(messages, ...)` — llama a create con modelo, max_tokens y la lista; devuelve `message.content[0].text` (o recorre bloques).

Un notebook posterior lo envolvió en un `while` (escribir hasta escape) como chatbot de consola. Los mismos auxiliares pueden ir detrás de FastAPI, Flask, Django o Streamlit. El modelo no conoce vuestra UI; solo ve el array que enviáis.

Contexto frente a «memoria» frente a RAG

En esta sesión, **contexto / ventana de contexto** son los tokens de esta petición: system + messages (+ tools luego). La «memoria» de conversación es lo que *vuestra app* persistió y volvió a meter en esa ventana. Cachear PDF dentro de Claude no es una base vectorial: Vijay dijo a quien preguntó que consultar una BD es **RAG** y es entrada recuperada, no almacén a largo plazo del modelo. No mezcléis las tres en el examen.

Prefill: mensajes assistant como arranque

Podéis colocar un mensaje assistant *antes* de que el modelo haya contestado, para que la generación continúe desde una semilla (por ejemplo una valla markdown). Eso es **prefill**, visto en profundidad el 19 de agosto. En este capítulo: el rol assistant es tanto «historia que rejugáis» como «respuesta parcial que queréis completar».

System prompt y alucinación

El notebook de system prompt fusionaba un `system` opcional en el diccionario de params de create. El propósito: fijar el rol y las reglas de proceso para que el modelo divague menos. No sustituye anclar con documentos, tools o RAG; es una loncha más del contexto de entrada.

UI DEL AULA

Cómo VS Code o Jupyter muestran un widget de chat no está en la API. Priman los nombres de campo de la **Messages API** si las firmas del notebook discrepan.

Ejercicios de examen

- Si la aplicación no reenvía turnos previos, el modelo no puede usarlos.
- El system prompt es un parámetro de create, no se guarda entre llamadas.
- User ≈ mensaje humano; assistant ≈ mensaje del modelo (o prefijado) en el array.
- Gestionar la lista es trabajo de aplicación; Claude Chat ya lo hace en el producto.

Documentación oficial

- **Messages API**

<https://docs.claude.com/en/api/messages>

Roles, system y ejemplos multi-turno.

- **Visión general de prompting**

<https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>

Cómo redactar system frente a contenido de usuario.

- **Visión general de la API**

<https://docs.claude.com/en/api/overview>

Construcción del cliente y forma de la petición.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.